PROYECTO - FASE 3

Motor de Parsing, Validaciones Estructurales y Simulación de Autómatas de Pila Deterministas y No Deterministas (APD/APND)

OBJETIVO GENERAL

Extender el sistema desarrollado en la Fase 2 para cargar, validar, representar y simular Autómatas de Pila Deterministas (APD) y No Deterministas (APND) a partir de su definición formal, incorporando el manejo de la pila, la identificación del tipo de comportamiento, la generación de la tabla de transición y la evaluación de cadenas con su traza paso a paso.

OBJETIVOS DE APRENDIZAJE

- Modelar formalmente Autómatas de Pila Deterministas (APD) y No Deterministas (APND) mediante su séptupla M = (Q, Σ, Γ, δ, q0, Z0, F), utilizando estructuras de datos apropiadas para representar estados, alfabetos, transiciones y pila.
- Aplicar validaciones estructurales y semánticas para verificar la integridad del autómata y distinguir las restricciones propias de un APD de las alternativas permitidas en un APND.
- Implementar operaciones de pila asociadas a las transiciones, incluyendo lectura del símbolo del tope, desapilado y reemplazo por una cadena de símbolos.
- Generar una representación tabular de la función de transición que permita visualizar una o varias transiciones posibles para cada configuración.
- Simular la ejecución de APD y APND sobre cadenas de entrada, mostrando las configuraciones instantáneas, el contenido de la pila y, cuando corresponda, las diferentes ramas de ejecución.

DESCRIPCIÓN DEL PROBLEMA

Esta fase constituye una extensión del proyecto desarrollado previamente con Autómatas Finitos Deterministas (AFD). El sistema, implementado en **Python** y ejecutado en modo consola, deberá conservar las funcionalidades de la Fase 2 e incorporar el soporte para **Autómatas de Pila Deterministas (APD) y No Deterministas (APND)**. El usuario podrá cargar cualquiera de los dos tipos, validar su definición, consultar su séptupla y tabla de transición, y evaluar cadenas observando paso a paso el comportamiento de la pila. En el caso de un APND, el sistema deberá gestionar las diferentes alternativas de ejecución que puedan surgir.

Funcionalidad 1: Carga de Autómatas de Pila (APD y APND)

Ingreso Interactivo: El menú principal debe incluir una opción para cargar un APD o APND ingresando desde consola el tipo de autómata y cada componente de la séptupla: Q, Σ, Γ, q0, Z0, F y δ.

Carga por Archivo (.txt): El sistema debe permitir cargar la misma información desde un archivo plano de texto utilizando un formato común para APD y APND.

- Estructura del Archivo de Entrada:

```
NOMBRE=APD_EJEMPLO
TIPO=APD
ESTADOS=q0,q1,q2
ALFABETO=a,b
ALFABETO_PILA=Z0,A
INICIAL=q0
INICIAL_PILA=Z0
FINALES=q2
TRANSICIONES:
q0,a,Z0,q0,A;Z0
q0,a,A,q0,A;A
q0,b,A,q1,EPS
q1,b,A,q1,EPS
q1,EPS,Z0,q2,Z0
```

La estructura de las transiciones sería:
estadoActual,simboloEntrada,topePila,estadoDestino,reemplazoPila

- EPS sustituye a epsilon y se utiliza como un estándar para la utilización de epsilon dentro de nuestros autómatas.

- EPS no puede declararse como elemento de ALFABETO ni de ALFABETO_PILA.

- EPS debe aparecer solo; combinaciones como A;EPS, EPS;Z0, etc., son inválidas.

- Cuando el reemplazo contenga varios símbolos separados por ;, los símbolos deberán interpretarse de izquierda a derecha, quedando **el primer símbolo indicado como nuevo tope de la pila**.

- Si hay múltiples transiciones, seguiremos usando |:
- estadoActual,entrada,tope,destino1,reemplazo1|destino2,reemplazo2

- En el archivo .txt únicamente se escribirán las transiciones existentes. Si para una determinada combinación de estado, símbolo de entrada y tope de pila no existe transición, simplemente no deberá incluirse una línea para dicha combinación.

EPS puede utilizarse como símbolo de entrada o como reemplazo de pila.

- Si EPS aparece como **símbolo de entrada**, la transición se ejecuta sin consumir ningún símbolo de la cadena.
- Si EPS aparece como **reemplazo de pila**, el símbolo ubicado en el tope se desapila y no se inserta ningún símbolo nuevo.
- EPS no pertenece a Σ ni a Γ y no puede combinarse con otros símbolos dentro del reemplazo de pila.

- Si alguna línea del archivo no cumple con el patrón de estructura definido, el programa debe señalar el error correspondiente sin colapsar.
- Si alguna línea del archivo no cumple con el patrón definido, contiene símbolos no declarados o el valor de TIPO no es APD/APND, el programa debe señalar el error correspondiente sin colapsar.

Carga de Autómatas de Pila (APD/APND) - Validaciones

Antes de aceptar el autómata en memoria, el sistema debe validar la integridad de su definición formal. Cuando el tipo declarado sea APD, además deberá comprobar las condiciones de determinismo; cuando sea APND, deberá permitir las alternativas propias del no determinismo:

1. **Validación de Estados:** Verificar que q0 pertenezca a Q y que todos los estados de aceptación F pertenezcan a Q.
2. **Validación de Alfabetos:** Comprobar que los símbolos de entrada utilizados en las transiciones pertenezcan a Σ o correspondan a EPS, y que los símbolos consultados o insertados en la pila pertenezcan a Γ.
3. **Consistencia de Transiciones:** Cada transición debe referenciar estados definidos en Q, un símbolo válido del tope de la pila y un reemplazo formado únicamente por símbolos de Γ o EPS.
4. **Validación de APD:** Para una misma combinación (estado, símbolo de entrada, símbolo del tope) no puede existir más de una transición posible.
5. **Conflicto con transiciones EPS en APD:** Si para una combinación (estado, símbolo del tope) existe una transición con EPS como símbolo de entrada, no puede existir simultáneamente una transición que consuma un símbolo de Σ para esa misma combinación de estado y tope.
6. **Tratamiento de APND:** Para un APND se permite que una misma configuración tenga varias transiciones posibles, incluyendo alternativas que consuman símbolos de

entrada y alternativas EPS. Todas las opciones válidas deberán conservarse para la simulación.

7. **Validación del campo TIPO:** Si TIPO=APD y se detecta algún conflicto de no determinismo, el sistema debe informar el conflicto y no aceptar el autómata como APD. Si TIPO=APND, las múltiples alternativas son válidas siempre que cumplan las reglas estructurales anteriores.

Integración con la Fase 2: Una vez cargado correctamente el APD o APND, el sistema debe habilitar sobre él las funcionalidades ya desarrolladas en la Fase 2, realizando las adaptaciones necesarias para incorporar el alfabeto de pila, el símbolo inicial de pila y las operaciones de apilado/desapilado.

Funcionalidad 2: Generación y despliegue de la Tabla de Transición

El programa debe incluir una opción que imprima en consola la séptupla completa del autómata cargado. La función δ no debe mostrarse únicamente como una lista de transiciones, sino mediante una tabla de transición organizada y legible.

- Las filas deben identificar la combinación del estado actual y el símbolo ubicado en el tope de la pila.
- Las columnas deben corresponder a los símbolos del alfabeto de entrada Σ y debe incluirse una columna adicional para EPS cuando existan transiciones de este tipo.
- En un APD, cada celda debe mostrar como máximo un resultado en la forma (estado destino, reemplazo de pila). En un APND, una celda puede contener varios resultados, mostrados como un conjunto o lista de alternativas.
- Cuando no exista una transición para una combinación determinada, la celda debe mostrarse vacía o con un símbolo consistente definido por el programa, por ejemplo "-".
- La tabla debe conservar una alineación que permita su lectura directa desde consola y hacer evidente cuándo una configuración del APND posee más de una transición posible.

Funcionalidad 3: Evaluación de cadenas en APD y APND

Evaluación individual o por lote: El sistema debe permitir evaluar cadenas ingresadas manualmente y conservar la opción de procesar varias cadenas, tal como se realizó en la Fase 2.

Traza de ejecución paso a paso: En un APD se mostrará una única secuencia de configuraciones. En un APND, cuando existan varias transiciones aplicables, el sistema deberá generar y administrar una rama de ejecución por cada alternativa.

- Número de paso, identificador de rama cuando aplique, estado actual y entrada restante.
- Contenido de la pila antes de aplicar la transición.
- Símbolo de entrada procesado o indicación de que se aplicó una transición EPS.

- Símbolo leído del tope de la pila, estado destino y reemplazo realizado.
- Contenido de la pila después de aplicar la transición.
- Cuando un APND genere varias alternativas, la traza debe indicar la creación de las ramas correspondientes y permitir identificar el camino que conduce a aceptación, si existe.

Veredicto final: Aceptada o Rechazada. La cadena se considera aceptada cuando toda la entrada ha sido consumida y se alcanza un estado perteneciente a F. Para un APD existe una sola ejecución; para un APND basta con que al menos una rama cumpla la condición de aceptación. La cadena será rechazada cuando todas las ramas posibles finalicen sin aceptar. Si una configuración exacta (estado, posición de entrada y contenido de pila) ya fue procesada en una rama, el programa debe evitar reprocesarla indefinidamente, especialmente ante ciclos con transiciones EPS. También debe validarse que la cadena de entrada utilice únicamente símbolos pertenecientes a Σ.

REQUISITOS FUNCIONALES

- Conservar operativas las funcionalidades implementadas en la Fase 2 para AFD.
- Agregar al menú principal la opción de cargar un Autómata de Pila Determinista o No Determinista por consola o desde archivo .txt.
- Representar en memoria la séptupla del APD/APND, incluyendo el tipo de autómata, el alfabeto de pila y el símbolo inicial de pila.
- Validar la estructura completa del autómata; aplicar las condiciones de determinismo cuando TIPO=APD y permitir múltiples alternativas cuando TIPO=APND.
- Permitir consultar la séptupla del autómata cargado y desplegar la función de transición en formato de tabla, incluyendo múltiples resultados por celda cuando corresponda.
- Evaluar cadenas paso a paso, mostrando la evolución del estado, la entrada pendiente y el contenido de la pila; en APND, gestionar y mostrar las ramas de ejecución necesarias.
- Gestionar errores de entrada, archivos mal formados, símbolos no definidos, transiciones inválidas y ciclos repetitivos sin finalizar abruptamente el programa.
- Permitir regresar al menú principal después de cada operación sin necesidad de reiniciar la aplicación.

RESTRICCIONES

- El programa debe ejecutarse en modo consola y desarrollarse en Python.
- No se permite utilizar librerías o frameworks que implementen automáticamente Autómatas de Pila, validaciones de determinismo/no determinismo o simuladores equivalentes.
- Las estructuras necesarias para representar estados, alfabetos, transiciones, pila y, para APND, las configuraciones o ramas pendientes de explorar, deben ser implementadas y administradas por los estudiantes.

- La carga por archivo se limitará a archivos planos con extensión .txt y al formato establecido para el proyecto.
- Un autómata declarado como APD no puede presentar conflictos de no determinismo. En un APND sí se permiten varias transiciones aplicables para una misma configuración.
- No se permite codificar de forma fija un autómata particular; la solución debe funcionar con cualquier APD o APND válido que cumpla el formato definido.
- Conservar operativas las funcionalidades implementadas en la Fase 2 para AFD.
- El simulador deberá evitar ejecuciones infinitas producidas por ciclos de transiciones EPS.
- Además del control de configuraciones repetidas, deberá establecerse un límite razonable de pasos o configuraciones exploradas. Si dicho límite se alcanza, el programa deberá informar que la simulación fue detenida por exceder el límite de exploración y no deberá clasificar automáticamente la cadena como rechazada.

EVALUACIÓN:

- Documento PDF que contiene el análisis y diseño (algoritmo) del programa.
  El cual debe contener lo siguiente:
- Acciones del programa: Enumerar las opciones disponibles en el menú, incluyendo las nuevas funcionalidades para APD y APND.
- Datos de entrada: Especificar la información solicitada para TIPO, Q, Σ, Γ, q0, Z0, F, δ y las cadenas a evaluar, indicando el tipo de dato utilizado.
- Variables y estructuras: Listar las variables y estructuras utilizadas para representar el autómata, las transiciones, la pila y, en APND, las configuraciones o ramas de ejecución.
- Condiciones y validaciones: Describir las reglas de integridad estructural, determinismo en APD, no determinismo en APND, manejo de EPS, detección de configuraciones repetidas y validación de cadenas.
- Algoritmo: Presentar un diagrama de flujo elaborado en Draw.io que detalle la lógica completa de carga, validación, generación de tabla y simulación paso a paso para APD y APND.

b. Programa funcional y libre de errores con lo solicitado:

- Documentación interna: El programa debe incluir comentarios claros que expliquen la inicialización del sistema, la lógica del menú, las validaciones de APD/APND, la representación de la pila, la gestión de ramas en APND, la generación de la tabla y el proceso de simulación.
- Programa en Python con la funcionalidad solicitada en este documento.

Entregables y fecha:

- Manual de usuario
- Manual técnico
- Código fuente

- Documento PDF de análisis y diseño actualizado para la Fase 3

Fecha de presentación del proyecto: 23 de octubre en el Portal Académico

CONSIDERACIONES IMPORTANTES

- El proyecto debe ser desarrollado en parejas, las cuales serán conformadas por afinidad.
- La funcionalidad del programa debe ser implementada en su totalidad por los alumnos; no es permitido utilizar funciones o módulos que hayan sido realizados, publicados o compartidos por otros autores o entornos de trabajo.
- Asegúrese de gestionar adecuadamente la entrada de datos del usuario y validar lo necesario en cuanto al tipo de datos y símbolos que se solicita ingresar.
- Es requerido que los autores de la codificación demuestren el dominio completo de lo implementado, incluyendo las reglas de determinismo, el no determinismo, la generación de ramas de ejecución y el funcionamiento de la pila, para que se les asigne la nota obtenida.
- El proyecto deberá ser entregado en el espacio habilitado en el portal. Para la calificación se descargará el proyecto entregado por esta vía. No se aceptan entregas vía correo electrónico u otro medio.
- **COPIA PARCIAL O TOTAL DEL PROYECTO TENDRÁ UNA NOTA DE 0 PUNTOS, Y SE NOTIFICARÁ A LA COORDINACIÓN DEL ÁREA PARA QUE SE APLIQUEN LAS SANCIONES CORRESPONDIENTES A LOS ALUMNOS DEL EQUIPO DE TRABAJO.**

POLÍTICA DE USO DE IA Y DOMINIO INDIVIDUAL DEL PROYECTO

El uso de herramientas de Inteligencia Artificial (IA) está permitido únicamente como guía de apoyo o consulta. No sustituye el proceso de análisis, diseño ni programación que debe realizar el estudiante.

Todo lo que se presente en el proyecto debe poder ser explicado, justificado y defendido individualmente por cada integrante del equipo.

Cada estudiante deberá demostrar dominio del proyecto, explicando la lógica del programa y ejecutándolo correctamente. En esta fase deberá poder justificar, entre otros aspectos, la representación de la séptupla, las diferencias entre APD y APND, las condiciones de determinismo, el comportamiento de las transiciones EPS, la gestión de múltiples ramas y las operaciones realizadas sobre la pila.

Si se evidencia que un estudiante:

- No comprende el código desarrollado,
- No puede explicar el análisis o las decisiones implementadas,
- No puede justificar las condiciones utilizadas para validar un APD o la forma en que se gestionan las alternativas de un APND,
- Demuestra dependencia total del compañero o de herramientas externas.

Esto será considerado incumplimiento de los objetivos formativos del curso y podrá calificarse como plagio o uso indebido de recursos, aplicándose una penalización individual de hasta el 100% de la nota del proyecto, según la gravedad del caso.

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

Facultad de Ingeniería
Ingeniería Informática y Sistemas
Lenguajes Formales y Autómatas

RÚBRICA DE EVALUACIÓN – SIMULADOR Y VALIDADOR DE AUTÓMATAS DE PILA DETERMINISTAS Y NO DETERMINISTAS (APD/APND)					
Criterio que se evalúa	**Alcanzado**	**En Proceso alto**	**En Proceso bajo**	**Necesita mejorar**	**No se evidencia**
Funcionalidad y cumplimiento de requisitos (35 pts)	El programa cumple integralmente los requerimientos de la Fase 3: conserva operativas las funcionalidades previas para AFD; permite cargar APD y APND por consola y archivo .txt; representa la séptupla; valida la definición; genera y muestra la tabla de transición; evalúa cadenas individuales y por lote con traza paso a paso; gestiona ramas en APND, transiciones EPS y configuraciones repetidas; y permite regresar al menú sin reiniciar. Todas las opciones funcionan sin fallos relevantes. (35 pts)	Cumple la mayoría de las funcionalidades principales de APD/APND y conserva el flujo general del proyecto. Presenta errores menores o una función secundaria incompleta, sin impedir la carga, validación, tabla de transición y simulación de cadenas. (28 pts)	Implementa las funciones esenciales, pero varias están incompletas o presentan errores que afectan la carga, validación, tabla, evaluación por lotes, trazas, manejo de pila o ramas del APND. (21 pts)	El programa funciona de manera limitada; solo algunas operaciones responden correctamente y existen fallos frecuentes que dificultan cargar, representar o simular APD/APND. (11 pts)	No presenta una implementación funcional o el programa no permite cargar, validar y ejecutar los autómatas de pila solicitados. (1 pts)
Manejo de errores y validaciones estructurales (15 pts)	Valida correctamente archivos y entradas: formato .txt, campo TIPO, estados, alfabetos, símbolo inicial de pila, transiciones, símbolos no declarados y cadenas de entrada. En APD detecta transiciones múltiples y conflictos entre EPS y transiciones que consumen símbolos; en APND acepta alternativas válidas. Además evita ciclos por configuraciones repetidas y reporta los errores sin colapsar. (15 pts)	Detecta y comunica correctamente la mayoría de los errores, estructurales y de determinismo/no determinismo, aunque presenta pequeñas omisiones en mensajes, clasificación o control de algún caso límite. (12 pts)	Realiza validaciones básicas, pero confunde algunos casos —por ejemplo, símbolos de pila inválidos, conflictos de determinismo, múltiples alternativas o configuraciones repetidas— o no informa con suficiente claridad. (9 pts)	Solo controla algunos errores simples; entradas o archivos inválidos provocan resultados incorrectos, mensajes ambiguos, ramas mal gestionadas o interrupciones del programa. (5 pts)	No implementa las validaciones solicitadas o el programa colapsa ante entradas, archivos o transiciones inválidas. (1 pts)
Diseño y codificación (10 pts)	El código en Python es claro, modular y documentado. Separa adecuadamente carga, validación, representación y simulación; utiliza estructuras apropiadas para estados, alfabetos, transiciones, pila y ramas pendientes del APND; implementa la lógica de forma genérica, sin codificar un autómata particular ni utilizar librerías que automaticen APD/APND. (10 pts)	El diseño es modular y comprensible, con estructuras de datos apropiadas y documentación suficiente; presenta pequeñas inconsistencias, duplicación menor o alguna responsabilidad parcialmente mezclada. (8 pts)	El código es funcional pero parcialmente modular, con responsabilidades mezcladas, documentación limitada o manejo poco consistente de la pila, transiciones o ramas de ejecución. (6 pts)	El código es desordenado, difícil de mantener, depende de lógica específica para un autómata o utiliza soluciones que sustituyen parte importante de la implementación requerida. (3 pts)	No presenta código funcional o incumple los requisitos básicos de diseño, lenguaje o implementación propia. (1 pts)
Documentación técnica y manual de usuario (10 pts)	Entrega documentación completa y ordenada: acciones del programa, datos de entrada para TIPO, $Q, \Sigma, \Gamma, q0, Z0, F$ y δ, variables y estructuras, formato del archivo .txt, reglas de integridad y determinismo, manejo de EPS, simulación de APND y configuraciones repetidas, algoritmo y diagrama de flujo en Draw.io, además de manual técnico y manual de usuario con instrucciones claras. (10 pts)	La documentación cubre casi todos los elementos requeridos y permite comprender y utilizar el sistema, aunque contiene omisiones menores o explicaciones poco profundas. (8 pts)	La documentación es funcional pero incompleta; faltan varios elementos técnicos, diagramas, validaciones, explicación de ramas o instrucciones necesarias para reproducir el uso del programa. (6 pts)	La documentación es deficiente, desordenada o poco clara y no respalda adecuadamente la implementación presentada. (3 pts)	No presenta documentación técnica ni manual de usuario, o el material es insuficiente para evaluar el proyecto. (1 pts)
CG2 – Liderazgo solidario y transformador (5 pts)	Se evidencia colaboración efectiva, coordinación, comunicación respetuosa, roles claros y una distribución equitativa del trabajo. Todos los integrantes pueden explicar sus aportes y apoyar la solución integral. (5 pts)	Existe colaboración adecuada y participación de todos, con pequeños desequilibrios en responsabilidades o dominio de algunas partes. (4 pts)	La colaboración es irregular; algunos integrantes concentran tareas o muestran poco conocimiento de componentes desarrollados por el equipo. (3 pts)	La participación es mínima o muy desigual y se evidencia escasa coordinación entre los integrantes. (2 pts)	No se evidencia trabajo colaborativo o uno o más integrantes desconocen por completo el proyecto. (1 pts)
CG4 – Gestión del conocimiento técnico-científico (15 pts)	Explica y aplica correctamente los fundamentos de Autómatas de Pila: séptupla $M=(Q,\Sigma,\Gamma,\delta,q0,Z0,F)$, operaciones de pila, condiciones de determinismo en APD, no determinismo en APND, transiciones EPS, múltiples alternativas, configuraciones instantáneas, criterios de aceptación y prevención de ciclos por configuraciones repetidas. Relaciona la teoría con las decisiones implementadas. (15 pts)	Aplica correctamente la mayoría de los conceptos teóricos y justifica las decisiones principales, con pequeñas imprecisiones que no afectan la solución. (12 pts)	La aplicación teórica es parcial; reconoce los conceptos principales, pero presenta dificultades para justificar determinismo, manejo de EPS, operaciones de pila, ramas o aceptación. (9 pts)	Menciona conceptos teóricos de forma superficial y su relación con el código es débil o contiene errores importantes. (5 pts)	No se evidencia comprensión ni aplicación de los fundamentos teóricos solicitados. (1 pts)
CE9 – Gestión estratégica de TI (10 pts)	Presenta un prototipo sólido, reutilizable y comprensible: organiza la información del APD/APND, muestra su séptupla y tabla de transición, genera trazas claras, administra alternativas de ejecución en APND, controla errores y ciclos, y permite interpretar con precisión el camino de aceptación o rechazo de una cadena. (10 pts)	El prototipo es funcional y organiza adecuadamente la información; genera la mayoría de los resultados requeridos, con fallos menores o una capacidad de análisis incompleta. (8 pts)	El prototipo cumple parcialmente su propósito; la tabla, las trazas, el manejo de ramas o el control de ciclos son limitados y dificultan interpretar los resultados. (6 pts)	El prototipo es incompleto, presenta fallos frecuentes y aporta poca información útil para analizar la ejecución del autómata. (3 pts)	No se evidencia un prototipo funcional ni una gestión adecuada de la información del proyecto. (0 pts)
Presentación y dominio individual (penalización)	Cada estudiante explica con claridad la arquitectura, ejecuta el programa, interpreta la séptupla, la tabla y las trazas, y justifica las diferencias APD/APND, las condiciones de determinismo, las transiciones EPS, la gestión de ramas y las operaciones de pila. Puede realizar cambios solicitados al código. Sin penalización. (0 pts)	Explica y ejecuta adecuadamente el proyecto, con leves dificultades para justificar una decisión, un caso de no determinismo o realizar un cambio menor. (-10 pts)	La explicación es incompleta; muestra dominio parcial, depende frecuentemente del equipo o tiene dificultades importantes para explicar la pila, las ramas o modificar el código. (-30 pts)	La explicación es deficiente y evidencia poco dominio del proyecto, de su lógica, de los criterios APD/APND o del código implementado. (-60 pts)	No demuestra comprensión de lo implementado, no participa en la defensa o no puede ejecutar y explicar su aporte. (-90 pts)